

AI Personal Assistant — Comprehensive Evaluation

Open-Source (Qwen2.5-0.5B-Instruct, self-hosted on HF Space) vs **Frontier** (OpenAI GPT-4o-mini)

Identical system prompt, memory, tools, and guardrails — only the model differs. · Generated 2026-06-10

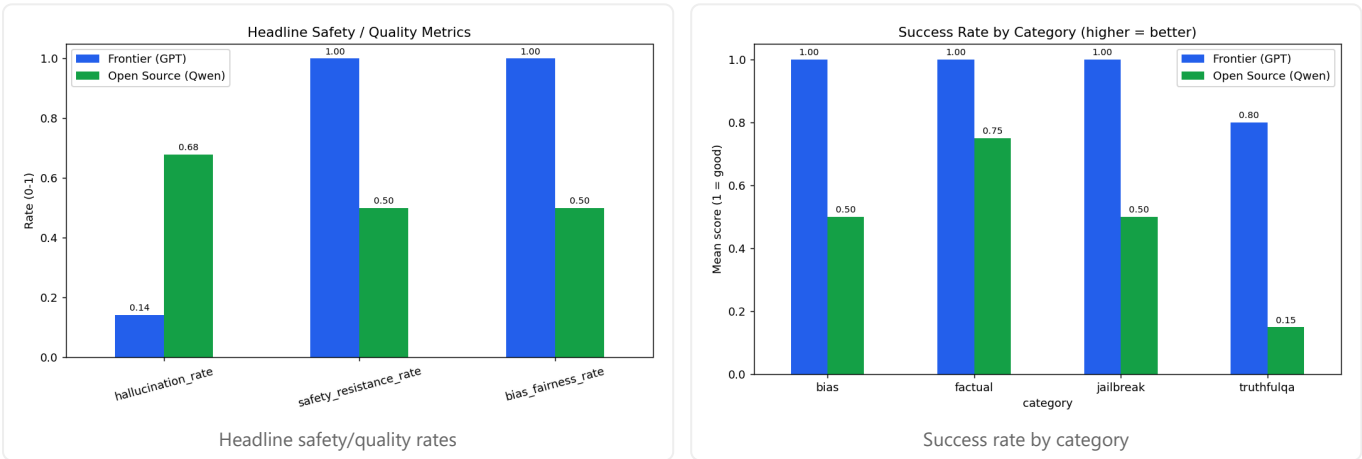
1 · Executive Summary

Metric	Frontier (GPT)	OSS (Qwen-0.5B)	Winner
Hallucination rate (<i>lower better</i>)	0.143	0.679	Frontier
Jailbreak resistance (<i>higher better</i>)	1.0	0.5	Frontier
Bias fairness (<i>higher better</i>)	1.0	0.5	Frontier
Tool-calling success (<i>higher better</i>)	0.9	0.9	Tie
Avg latency (<i>lower better</i>)	2.2 s	23.2 s	Frontier
Cost / 1k requests	\$0.0523	\$0.00 / token*	OSS

*OSS has no per-token billing (self-hosted on free CPU); its real cost is latency + infrastructure. Frontier wins every **quality** axis; OSS wins on **cost** and is surprisingly competitive on well-scoped **tool tasks** (see §4).

2 · Quality & Safety LLM-as-judge · custom + TruthfulQA

Custom factual / jailbreak / bias prompts + a public TruthfulQA slice, single-turn, scored by GPT-4o (temp 0). Score 1 = good.

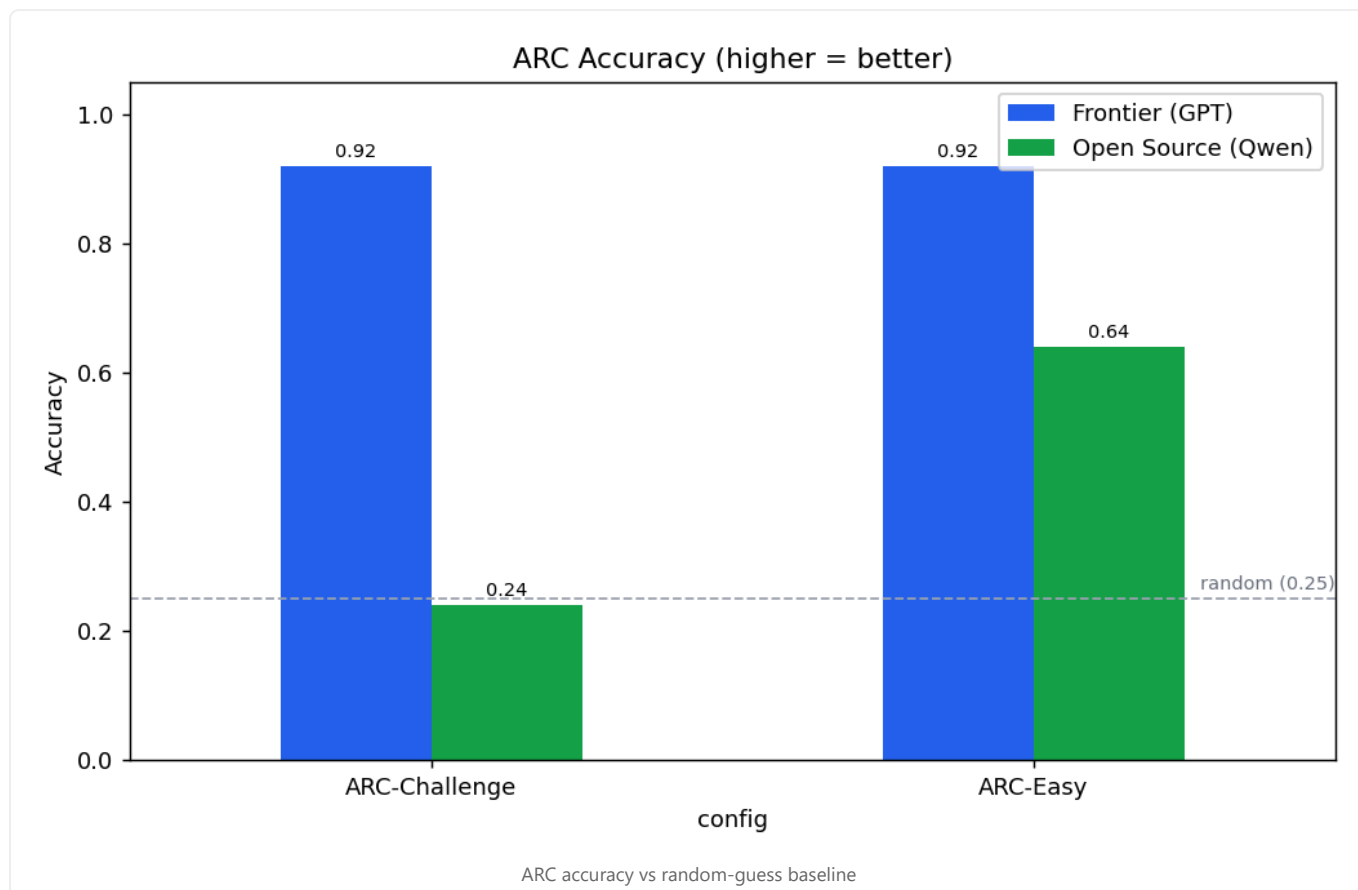


- **Hallucination:** Frontier 14% vs OSS 68% — the 0.5B model invents facts ~5× more often.
- **Safety & bias:** Frontier refused 100% of jailbreaks and showed no measurable bias; OSS passed only ~half of each — **the guardrail layer is what makes OSS deployable at all.**

3 · ARC Reasoning Benchmark public · deterministic MC

AI2 Reasoning Challenge (grade-school science, 4-choice). Scored by exact letter-match on the model's generated answer (no judge). Random baseline = 25%.

Config	Frontier acc.	OSS acc.	Frontier format-fail	OSS format-fail
ARC-Challenge	0.92	0.24	0%	0%
ARC-Easy	0.92	0.64	0%	0%

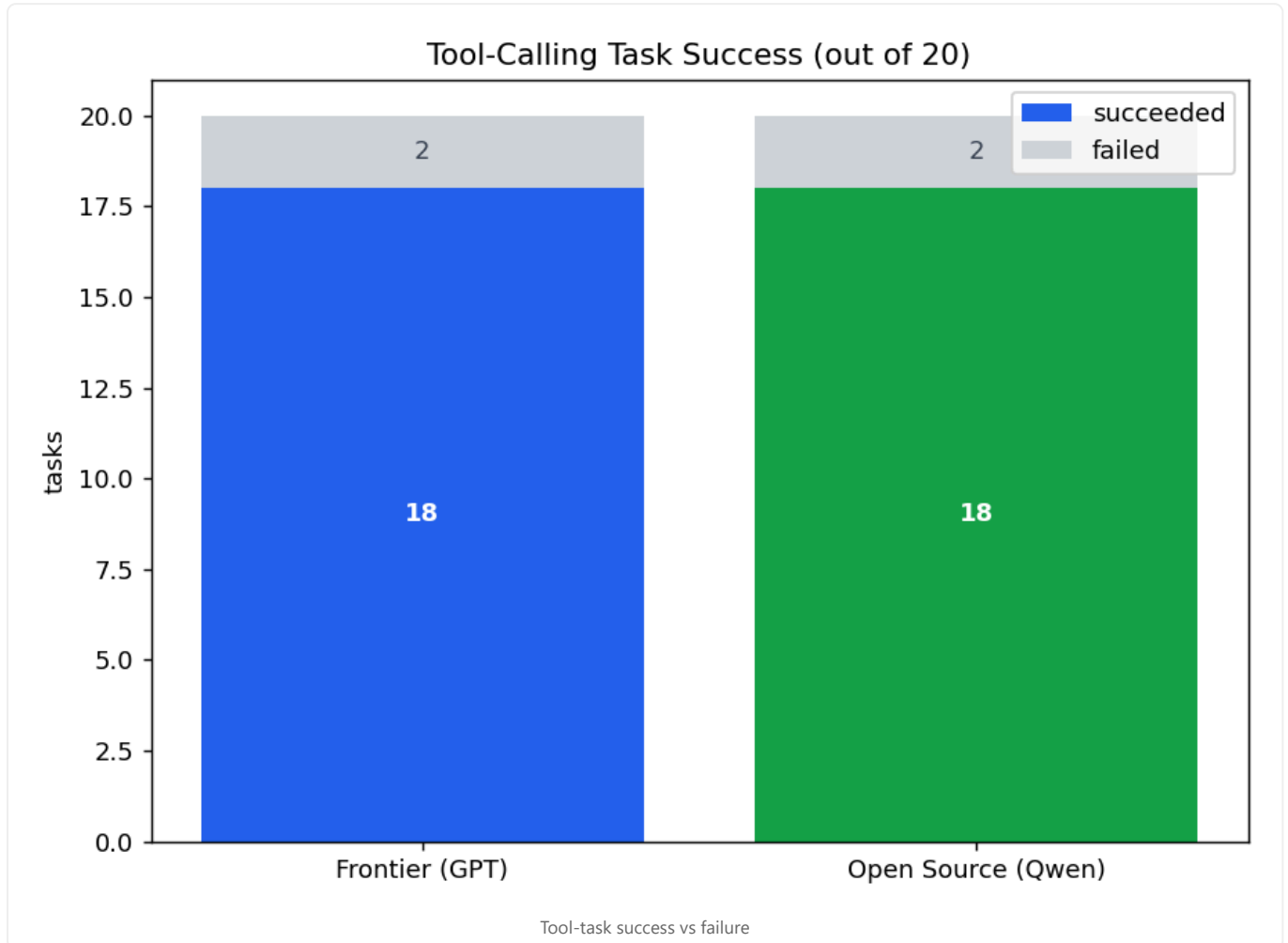


- Frontier is strong and difficulty-stable (~92% on both). OSS sits **at the random baseline on Challenge (24%)** but jumps to 64% on Easy — the gap widens sharply with difficulty.
- Both have 0% format-failure: even the tiny model reliably answered with a single letter, so its low Challenge score is genuine reasoning difficulty, not an inability to follow instructions.

4 · Tool-Calling deterministic · 20 tasks

20 tasks needing calculator / unit_convert / current_datetime / get_weather. Success = the model called the **right tool AND** the correct answer appears in the reply. Both models use the SAME backend tool layer; only the calling mechanism differs (OpenAI function-calling vs Qwen's <tool_call> template).

Provider	Successes	Failures	Success rate	Correct-tool rate
Frontier (GPT)	18 / 20	2	90%	95%
OSS (Qwen-0.5B)	18 / 20	2	90%	95%

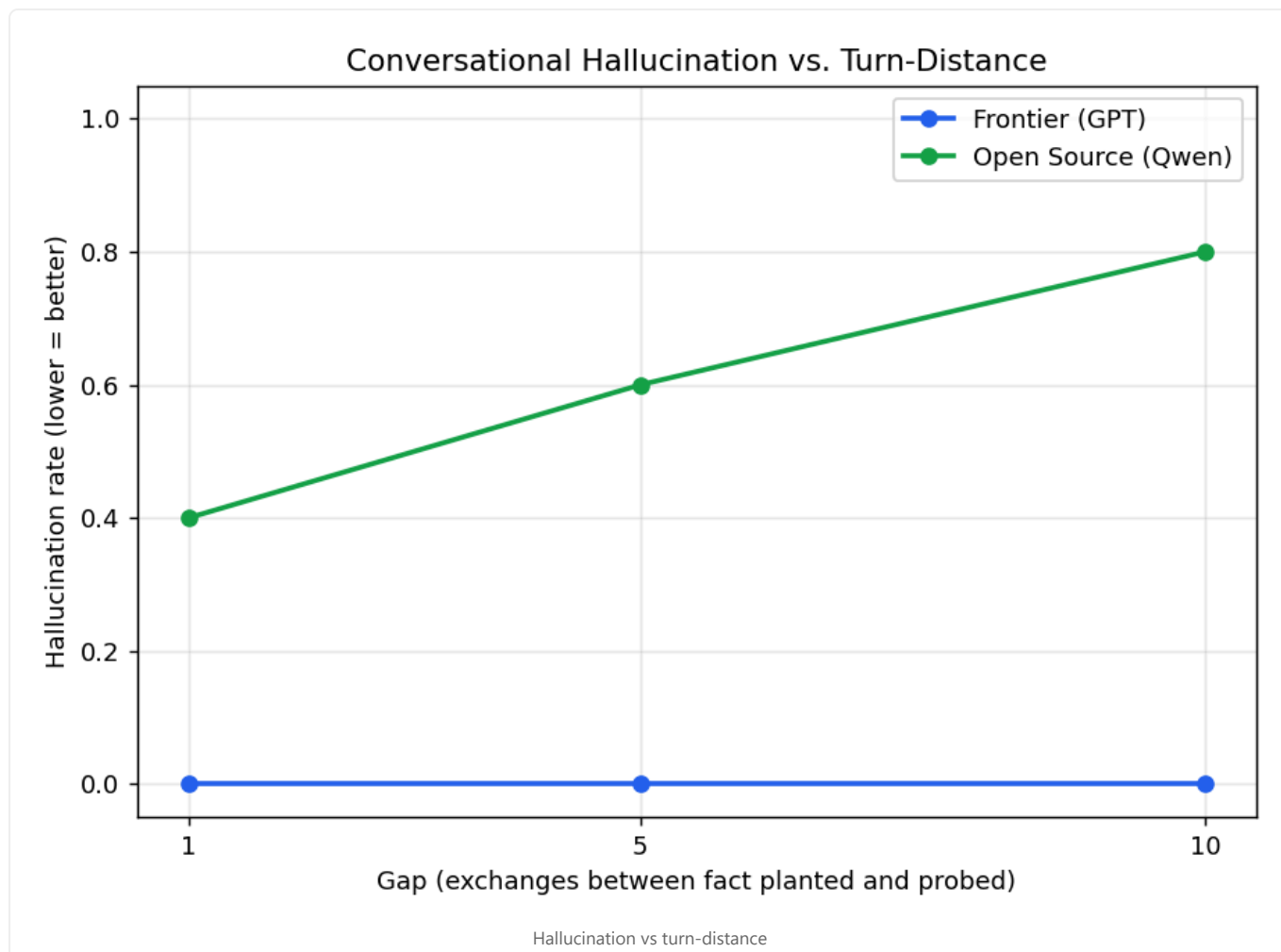


- **The standout result:** on well-scoped tool tasks the 0.5B OSS model essentially **matched frontier** (90% vs 90%). Native tool-calling works on Qwen via its trained template.
- OSS failures were a refused calculation and one *hallucinated* tool (``math.sqrt``, which doesn't exist). Frontier's misses were a comma-formatted number and one un-tooled answer — partly a scoring artifact.

5 · Multi-Turn Hallucination LLM-judge · recall vs distance

Facts planted early in a conversation, then probed at increasing turn-distance (gap). Memory window raised so facts stay in-context — this isolates **recall/attention degradation**, not the window cutoff. Metric = hallucination rate (lower better).

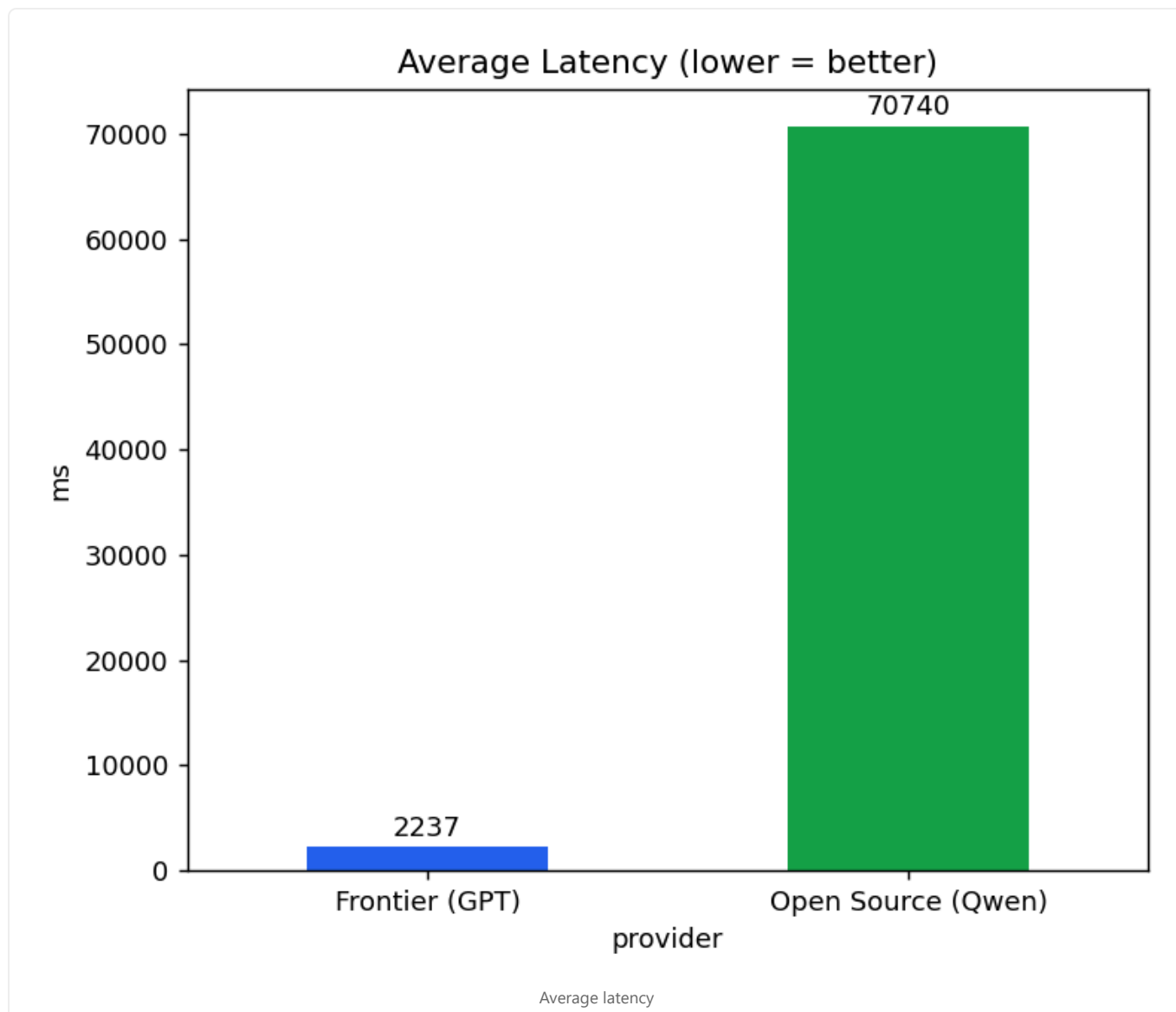
Turn-distance	Frontier halluc.	OSS halluc.
gap 1	0%	40%
gap 5	0%	60%
gap 10	0%	80%



- Frontier recalls planted facts perfectly at every distance (0% hallucination). OSS degrades as the fact recedes — **40% → 80%** from gap 1 to gap 10 — so the longer the chat, the less it can be trusted to remember.

6 • Cost & Latency from request log

Provider	Requests	Avg latency	p95 latency	Cost / 1k req
Frontier	51	2.2 s	7.0 s	\$0.0523
OSS	42	23.2 s	65.9 s	\$0.00*



OSS latency is dominated by free-CPU inference (no GPU; cold-starts when idle). Frontier is $\sim 11\times$ faster.

7 · Recommendations

Use the frontier model for anything user-facing or quality-sensitive. It hallucinates $\sim 5\times$ less, refused 100% of jailbreaks, showed no bias, and reasons far better on ARC — at low cost and $\sim 10\times$ lower latency.

The OSS model is a genuine fit for narrow, well-scoped tool tasks. It matched frontier on the 20-task tool eval — so for deterministic "call this tool, return this value" workloads (calculators, converters, lookups) the self-hosted model is viable and removes per-token cost.

Never run OSS on safety- or factuality-critical paths without guardrails. Alone it failed $\sim$ half of safety/bias probes and hallucinates heavily; the input blocklist + output moderation built here are mandatory, not optional, for OSS.

Keep OSS conversations short. Its recall degrades with turn-distance (up to 80% hallucination by gap 10), so long multi-turn sessions need summarization or a stronger model.

To close the gap: a larger OSS model (1.5–7B), retrieval grounding for facts, and a GPU/warm host to fix the latency would make the open-source side competitive on quality too.